

Deterministic Pre-Oracle Filtering for Quantum Oracle Sketching via Modular Wheel Constraints

Dan Hawkley

Abstract

Motivated by Quantum Oracle Sketching (QOS), we introduce a deterministic pre-oracle filtering layer based on modular wheel constraints (mod30, mod210, mod2310). The layer reduces raw classical input before feature construction, sampling, or oracle-preparation stages. Across the tested wheel filters, candidate streams are reduced by approximately 73–79% while preserving usable downstream classifier behavior. Compared with matched random subsampling, wheel filtering exhibits comparable behavior while remaining deterministic and reproducible. A QOS-style wrapper experiment shows reduced preprocessing cost, and a row-order robustness check indicates that results are not driven by accidental dataset ordering. This work does not modify QOS internals or claim quantum advantage; it studies a simple classical front-end layer for reducing the input stream entering QOS-style pipelines.

1 Introduction

Zhao’s Quantum Oracle Sketching (QOS) framework studies how massive classical data can be processed through quantum-accessible sketches. The present work studies a complementary front-end question:

Before QOS oracle construction begins, can a deterministic classical filter reduce the raw input stream while preserving downstream behavior?

We answer this question experimentally using modular wheel constraints. The proposed layer is applied before feature construction or downstream processing, and it requires no modification to QOS internals.

The core contribution is a deterministic pre-oracle filtering layer that:

- reduces raw candidate streams before QOS-style processing;
- preserves usable downstream classifier behavior;
- behaves comparably to matched random subsampling;
- remains stable under dataset reordering.

Code Availability Code and experiments are available at: github.com/thinkthoughts/quantum-oracle-sketching-mod30.

The modwheel filtering branch is: github.com/thinkthoughts/quantum-oracle-sketching-mod30/tree/modwheel layer.

2 QOS Motivation and Practical Impact

QOS-style pipelines begin from classical data streams. Any reduction before oracle preparation, sketching, or downstream feature construction may reduce classical preprocessing cost.

The proposed wheel layer acts as:

```
raw classical input
-> deterministic wheel filter
-> reduced input stream
-> feature construction / QOS-style processing
-> downstream task
```

In the experiments below, this front-end layer reduces candidate streams by approximately 73–79%. The main practical observation is:

less input enters the pipeline, preprocessing cost decreases, and downstream behavior remains usable.

This makes the method relevant as a QOS-compatible front-end filter, even though it does not alter the QOS algorithm itself.

3 Experiments

3.1 QOS-Style Wrapper: Filtering Before Feature Construction

The most QOS-relevant test applies filtering before feature construction. This reflects the intended position of the method: upstream of oracle preparation or downstream feature construction.

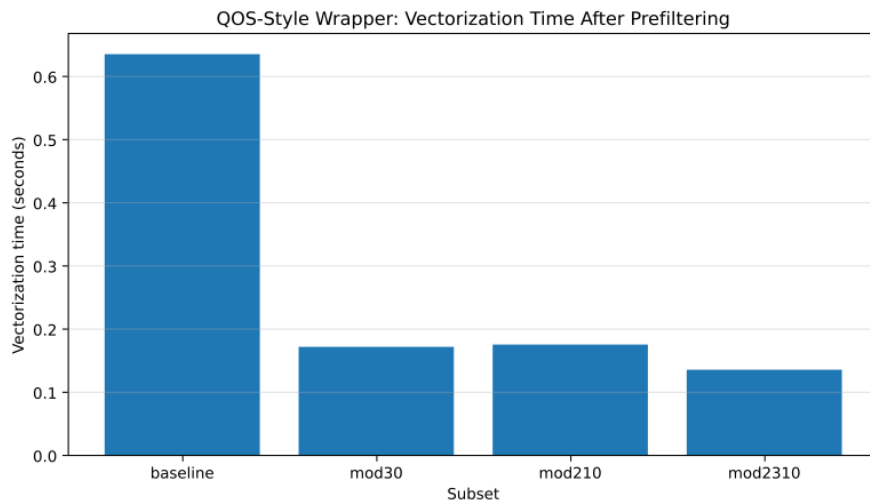


Figure 1: QOS-style wrapper experiment. Vectorization time decreases when the raw input stream is filtered before feature construction.

Figure 1 shows the primary systems-level benefit: reducing the raw input stream reduces preprocessing workload. This is the closest experiment to the intended QOS front-end placement.

3.2 Downstream Stability After Prefiltering

A useful pre-oracle filter should reduce input size without destroying downstream task behavior.

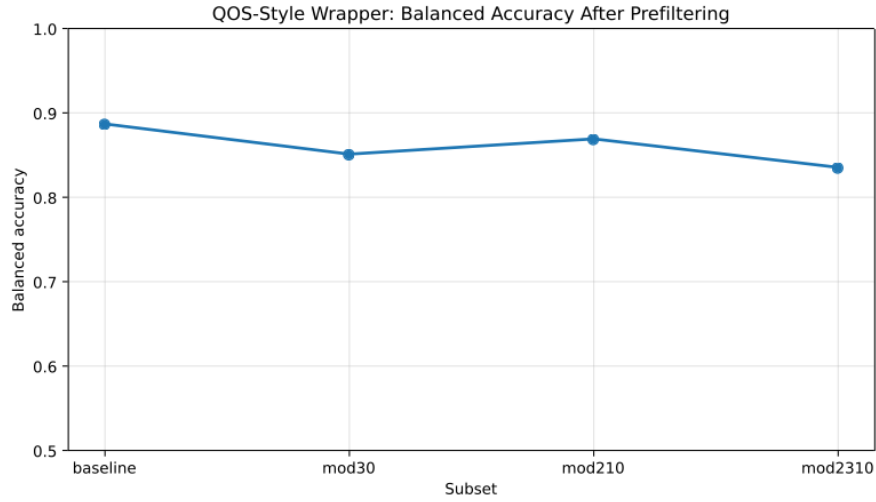


Figure 2: Balanced accuracy in the QOS-style wrapper experiment. Accuracy remains usable across filtering levels.

Figure 2 shows that downstream classifier behavior remains stable across the tested filtering levels. This supports the claim that the wheel layer can reduce raw input while preserving usable downstream behavior.

3.3 Comparison with Matched Random Subsampling

A natural baseline is random subsampling at the same retained fraction. We therefore compare deterministic wheel filtering against matched random subsets.

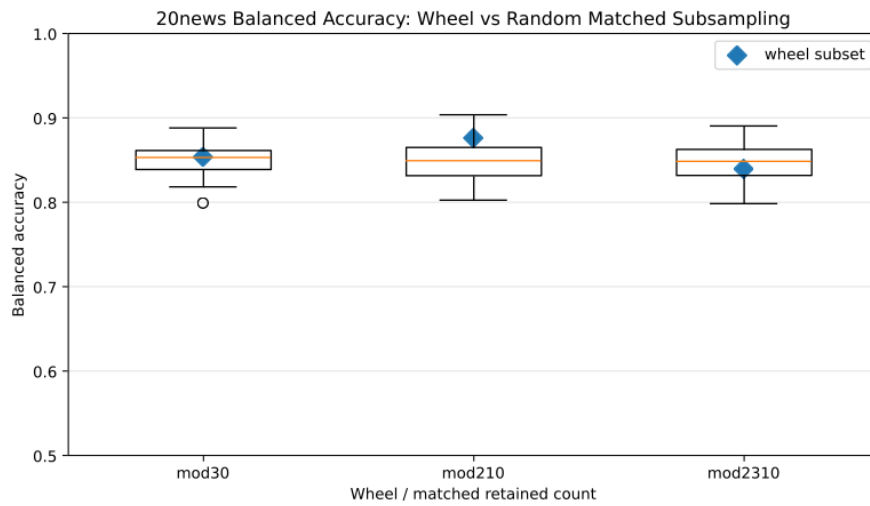


Figure 3: Wheel filtering compared with random subsampling at matched retained fractions. Wheel-filtered results lie within the distribution of random matched subsets.

Figure 3 shows that wheel filtering behaves comparably to matched random subsampling. The value of the wheel layer is therefore not a magic accuracy improvement; it is a deterministic, reproducible alternative to random reduction before QOS-style processing.

3.4 Robustness to Dataset Ordering

Because row-ID filters can be sensitive to dataset ordering, we test whether results change after randomly permuting the input rows.

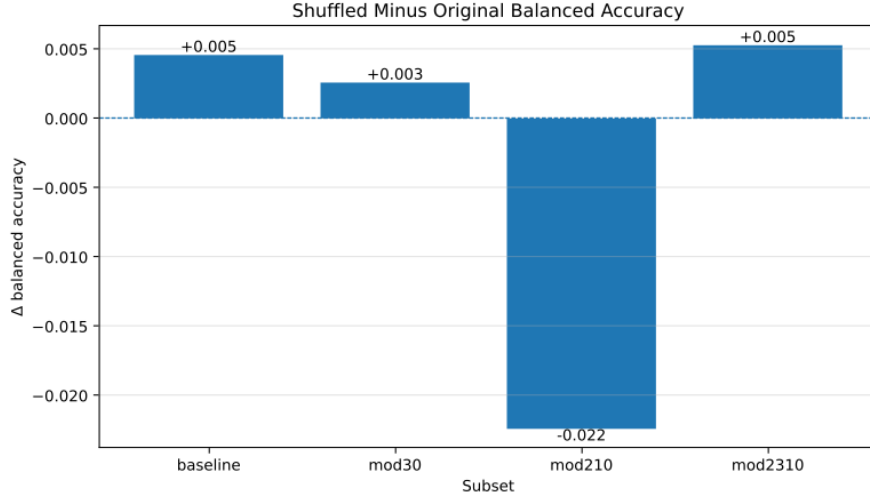


Figure 4: Robustness to dataset ordering. The figure reports shuffled-minus-original balanced accuracy. Differences remain small.

Figure 4 indicates that the observed behavior is not primarily driven by accidental row ordering. This strengthens the interpretation of the wheel layer as a stable preprocessing adapter.

4 Method: Modular Wheel Filtering

A modular wheel filter is defined by a modulus built from small primes. For a modulus M , an index or candidate residue r is retained when

$$\gcd(r, M) = 1.$$

We evaluate three filters:

Wheel	Modulus	Residues	Candidate Fraction
mod30	30	8	0.2667
mod210	210	48	0.2286
mod2310	2310	480	0.2078

Table 1: Wheel filters used in the experiments. Candidate fraction gives the theoretical fraction of retained indices.

These filters retain approximately 20–27% of candidate indices and therefore remove approximately 73–79% of the raw input stream.

5 Adapter Design

The filter is intentionally non-invasive. It can be placed before QOS-style preparation as an external wrapper:

```
texts, labels
-> apply_modwheel_text_prefilter(...)
-> filtered_texts, filtered_labels
-> feature construction / QOS preparation
-> downstream task
```

This is a front-end adapter, not a replacement for QOS. It can be toggled on or off without changing core QOS routines.

6 Discussion

The experiments support a narrow but useful claim: deterministic wheel filtering can reduce the raw input stream before QOS-style processing while preserving usable downstream behavior.

The strongest result is the QOS-style wrapper experiment, which applies filtering before feature construction and shows reduced preprocessing time. The random-subset comparison clarifies that the method behaves similarly to ordinary subsampling at matched size. The ordering robustness experiment addresses the concern that row-ID filtering might depend on accidental row order.

This work does not claim:

- quantum advantage;
- improved QOS complexity;
- improved classifier accuracy;
- modification of QOS internals.

Instead, it proposes and tests a deterministic classical front-end filter that may reduce the amount of data entering QOS-style workflows.

7 Conclusion

We presented a deterministic pre-oracle filtering layer for QOS-style pipelines using modular wheel constraints. The filter reduces raw input streams by approximately 73–79%, reduces preprocessing cost in a wrapper experiment, behaves comparably to matched random subsampling, and remains stable under dataset reordering.

This provides a simple, reproducible, non-invasive preprocessing component for exploring classical front-end reductions before QOS oracle construction or related data-intensive workflows.

References

H. Zhao, *Exponential Quantum Advantage in Processing Massive Classical Data*, arXiv:2604.07639, 2026.